

数据的容器

数组与线性存储 · 12题 · C++ & Python 双语对照

小鲁最近发现一个难题：他想统计全班50个同学的成绩，难不成要写50个变量？"这也太麻烦了吧！"他抱怨道。小华走了过来："你需要的是**数组**——编程中最基本的数据容器。"

本章12道题，小鲁将在小华（数学天才）和小栋（工程师）的帮助下，从基础的一维数组操作，逐步掌握二维矩阵、递推算法和算法优化。每一题都是一个故事——编程不应该是枯燥的，它应该像厦大芙蓉湖畔的凤凰花一样美丽。

本章角色

-  **小鲁**（初学者）· 代表正在学习编程的你，会犯错、会提问、会进步
-  **小华**（算法高手）· 数学天才，ACM竞赛队成员，总能用简单的话解释复杂算法
-  **小栋**（工程师）· 喜欢用编程解决实际问题，数据分析爱好者
-  **小嘉**（校主精神）· 厦大创始人，偶尔现身，带来数学智慧与校训力量

NQ043 数组替换 AcWing 737

小鲁正在写一个数据处理程序，但发现数据里混入了一些负数和零——这些"脏数据"会影响后面的计算。"得想个办法，把不合格的数据统一替换成1....."小鲁嘀咕着。小华凑过来看了一眼屏幕："这不就是数组遍历+条件判断吗？遍历每个元素，如果 ≤ 0 就改成1，一行代码的事。"

- 输入** 10行，每行一个整数。
- 输出** 10行，每行" $X[i] = Y$ "。
- 范围** $-10^9 \leq \text{值} \leq 10^9$

输入	输出
0	$X[0] = 1$
-5	$X[1] = 1$
63	$X[2] = 63$
-8	...
0	
1	
2	
-100	
50	
200	

思路 这是数组最基础的操作模式：读入→判断→输出。遍历每个元素时检查其值，如果 ≤ 0 就替换为1。
C++中用for(int i=0;i<10;i++)访问，Python用enumerate()同时获取索引和值。

技巧 Python的enumerate(arr)返回(index, value)元组。C++中数组索引从0开始——这是初学者最常犯的off-by-one错误来源。记住：第0个位置是第一个元素。

C++

```
#include <iostream>
using namespace std;
#define For(a,begin,end) for(int a =begin; a<end;a++)
int main(){
    int x[10];
    For(i,0,10) cin>>x[i];
    For(i,0,10) if (x[i]<=0) x[i]=1;
    For(i,0,10) cout<<"X["<<i<<" ] = "<<x[i]<<endl;
    return 0;
}
```

Python

```
for i in range(10):
    x = int(input())
    if x <= 0: x = 1
    print(f"X[{i}] = {x}")
```

NQ044 数组填充 AcWing 738

小华在教小鲁数组的递推生成："你看，这不就像我们小时候学过的'翻倍'游戏吗？第0个是V，第1个是V×2，第2个是(V×2)×2..."小鲁恍然大悟："原来数组可以这样自动生成！不用我一个一个敲数字了。"

输入 一个整数V。

输出 10行，每行"N[i] = X"。

范围 $-10^4 \leq V \leq 10^4$

输入

1

输出

N[0] = 1
N[1] = 2
N[2] = 4
N[3] = 8
...

思路 递推生成：每个新值由前一个值×2得出。这是"递推思想"的入门——每个新结果基于之前的结果计算，不需要从头算起。Python列表推导：[v*(2**i) for i in range(10)]一行搞定。

技巧 Python的一行列表推导：arr = [v * (2**i) for i in range(10)]。C++中i从0开始，v每次乘2。注意V为负数时序列呈负增长。

C++

```
#include <cstdio>

int main()
{
    int n[10];
    int v;
    scanf("%d", &v);
    n[0] = v;

    for (int i = 1; i < 10; i++) n[i] = n[i - 1] * 2;

    for (int i = 0; i < 10; i++) printf("N[%d] = %d\n",
i, n[i]);

    return 0;
}
```

Python

```
v = int(input())
for i in range(10):
    print(f"N[{i}] = {v}")
    v *= 2
```

NQ045 数组选择 AcWing 739

小鲁看着屏幕上的100个数据犯了愁："这么多数据，我怎么能快速找出哪些小于等于10的？"小华笑道："这不就是'筛选'吗？Python一行代码就能搞定——列表推导或者边读边判断。记住，你不需要把所有数据都存下来，读一个判断一个就行。"

- 输入** 100行（或一行多个），每行一个浮点数。
- 输出** 对每个 ≤ 10 的数，输出" $A[i] = X$ "（保留1位小数）。
- 范围** $-10^6 \leq \text{值} \leq 10^6$

输入

0
-5
63
8.5
100
...

输出

$A[0] = 0.0$
 $A[1] = -5.0$
 $A[3] = 8.5$
...

思路 流式处理：不需要把100个数都存下来，读一个判断一个输出一个。这种"边读边处理"的思维在处理海量数据时至关重要——内存是有限的，流式处理可以处理任意大小的数据。

技巧 Python格式化f" $A[{i}] = {x:.1f}$ ". C++用printf(" $A[{}d] = {:.1f}\n$ ", i, x)。条件用 $\leq$ ，注意浮点精度。

C++

```
#include <cstdio>

int main()
{
    double a[100];

    for (int i = 0; i < 100; i ++ ) scanf("%lf", &a[i]);

    for (int i = 0 ; i < 100; i ++ )
        if (a[i] <= 10)
            printf("A[%d] = %.1lf\n", i, a[i]);

    return 0;
}
```

Python

```
for i in range(100):
    x = float(input())
    if x <= 10:
        print(f"A[{i}] = {x:.1f}")
```

NQ046 数组中的行 AcWing 743

小鲁在整理12个月的销售数据，每个月的销售额存在不同的行里。"我想单独看某个月的总销售额....."小华点点头："这就是二维数组的行操作。外层循环控制行，内层循环控制列，当行号匹配时累加。"

输入 第一行L(0~11)。第二行T('S'或'M')。接下来144个浮点数（12×12矩阵）。

输出 结果保留1位小数。

范围 矩阵元素 -10^6 到 10^6

输入

2
S
(144个浮点数...)

输出

(第2行之和)

思路 双层循环是二维数组的标准遍历方式。外层for i in range(12)控制行，内层for j in range(12)控制列。当i==L时累加。这种行列遍历模式贯穿所有多维数组题目。

技巧 Python嵌套列表：[[0]*12 for _ in range(12)]。注意不能用[[0]*12]*12——那是浅拷贝，所有行指向同一个列表！

C++

```
#include <cstdio>

int main()
{
    double a[12][12];

    int l;
    char t;
    scanf("%d\n%c", &l, &t);

    for (int i = 0; i < 12; i ++ )
        for (int j = 0; j < 12; j ++ )
            scanf("%lf", &a[i][j]);

    double s = 0;
    for (int i = 0; i < 12; i ++ ) s += a[l][i];

    if (t == 'S') printf("%.1lf\n", s);
    else printf("%.1lf\n", s / 12);

    return 0;
}
```

Python

```
l = int(input())
t = input().strip()
sum_val = 0
for i in range(12):
    for j in range(12):
        x = float(input())
        if i == l: sum_val += x
print(f"{sum_val if t == 'S' else sum_val / 12:.1f}")
```

NQ047 数组变换 AcWing 740

小鲁想把20个数据"倒过来看"——第一个放最后，最后一个放最前。"这不就是翻转吗？"小华说，"Python用arr[::-1]一行搞定，C++要手写交换循环。但理解算法比会用库更重要——你知道翻转的原理是什么吗？"

输入

20个整数。

输出

20行，每行"N[i] = X"。

范围

 $-10^9 \leq \text{值} \leq 10^9$

输入

```
0
1
2
...
19
```

输出

```
N[0] = 19
N[1] = 18
...
```

思路 对称交换算法：arr[i]与arr[19-i]交换，只需循环10次（i从0到9）。每步交换两个对称位置的元素。这是理解"手写翻转"和"调用库函数"差异的好例子。

技巧 Python翻转用arr[::-1]（切片反转）或arr.reverse()（原地反转）。C++用std::reverse()或手写swap。切片创建新列表消耗额外内存。

C++

```
#include <cstdio>
#include <iostream>

using namespace std;

int main()
{
    int c[20], d[20];

    for (int i = 0; i < 20; i++) cin >> c[i];
    for (int i = 19, j = 0; i >= 0; i--, j++) d[j] = c[i];

    for (int i = 0; i < 20; i++) printf("N[%d] = %d\n", i, d[i]);

    return 0;
}
```

Python

```
arr = [int(input()) for _ in range(20)]
arr.reverse()
for i, v in enumerate(arr):
    print(f"N[{i}] = {v}")
```

NQ048 斐波那契数列 AcWing 741

小鲁在图书上看到了斐波那契数列：0,1,1,2,3,5,8,13... "这个数列有什么神奇的吗？"小华兴奋地说："太多了！兔子的繁殖、花瓣的排列、黄金分割.....在编程里，它是最经典的递推题——每个数都是前两个数的和，这就是动态规划(DP)的萌芽！""不过要注意，第60项已经超过20亿了，C++要用long long。"

输入

一个整数N。

输出

一行，空格隔开的N个整数。

范围

 $1 \leq N \leq 60$

输入

5

输出

0 1 1 2 3

思路 递推标准模板：a=0,b=1;不断输出a，然后a,b=b,a+b。Python的a,b=b,a+b是最优雅的递推写法。C++必须用long long（64位），N=60时 $F(60) \approx 1.5 \times 10^{12}$ 远超出int范围。

技巧 C++千万用long long！int在N>46时就会溢出。Python自动大整数。a,b=b,a+b是Python递推的"指纹"——看到这行代码就知道在做递推。

C++

```
#include <iostream>
#include <cstdio>

using namespace std;

int main()
{
    long long f[61];
    f[0] = 0; f[1] = 1;

    for (int i = 2; i <= 60; i++) f[i] = f[i - 1] + f[i - 2];

    int n;
    cin >> n;
    while (n-- )
    {
        int x;
        cin >> x;
        printf("Fib(%d) = %lld\n", x, f[x]);
    }

    return 0;
}
```

Python

```
n = int(input())
a, b = 0, 1
result = []
for _ in range(n):
    result.append(str(a))
    a, b = b, a + b
print(' '.join(result))
```

NQ049 最小数和它的位置 AcWing 742

小栋找小鲁帮忙："我在做数据分析，找一组测量数据中的最小值以及它出现的位置。"小鲁打开TRAE，写下了min()和index()。小华看了说："不错，但你知道吗——你这实际上遍历了两遍数据。更好的方法是只遍历一次，同时记录最小值和位置。"

输入

第一行N。第二行N个整数。

输出

第一行"Menor valor: X"。第二行"Posicao: Y"（从0开始）。

范围

$1 \leq N \leq 1000$

输入

10
1 2 3 4 -5 6 7 8 9 10

输出

Menor valor: -5
Posicao: 4

思路 一次遍历同时找最小值和位置：初始设第一个值为最小值，后续每遇到更小的就更新。Python用min()+index()简洁但需要两次遍历。手写版本只需一次O(n)——理解算法效率的差异。

技巧 Python的min(arr)和arr.index(mn)组合虽简洁但效率不是最优。手写版本通过单次遍历实现O(n)时间+O(1)空间，是算法优化的入门案例。

C++

```
#include <cstdio>
#include <iostream>

using namespace std;

int main()
{
    int a[1001];
    int n;

    cin >> n;
    for (int i = 0; i < n; i++) cin >> a[i];

    int p = 0;
    for (int i = 1; i < n; i++)
        if (a[i] < a[p])
            p = i;

    printf("Minimum value: %d\n", a[p]);
    printf("Position: %d\n", p);

    return 0;
}
```

Python

```
n = int(input())
arr = list(map(int, input().split()))
mn = min(arr)
print(f"Menor valor: {mn}")
print(f"Posicao: {arr.index(mn)}")
```

NQ050 数组中的列 AcWing 744

"刚才你帮我算了某行的总和，现在我想算某列的总和——比如第3列，代表3月份各产品的销售额。"小栋说。小鲁看了一下代码："噢，这不就是把*i*==*L*改成*j*==*C*吗？其他都一样。"小华赞许地点点头："这就是编程的对称思维——理解了关系表达式，行和列的操作本质上是对称的。"

输入 第一行列号*C*。第二行操作类型*T*。接下来144个浮点数。

输出 结果保留1位小数。

范围 同NQ046

输入

2
S
(144个浮点数...)

输出

(第2列之和)

思路 与行操作对称：判断条件从*i*==*L*改为*j*==*C*。这种对称性是理解多维数组遍历的关键——能灵活变换遍历条件才是真正的掌握。

技巧 对比NQ046：代码几乎一样，只改变了一个字母。行列对称是编程美学的体现。

C++

```
#include <iostream>
#include <cstdio>

using namespace std;

int main()
{
    int c;
    char t;
    double m[12][12];

    cin >> c >> t;
    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++)
            cin >> m[i][j];

    double s = 0;
    for (int i = 0; i < 12; i++) s += m[i][c];

    if (t == 'S') printf("%.11f\n", s);
    else printf("%.11f\n", s / 12);

    return 0;
}
```

Python

```
c = int(input())
t = input().strip()
sum_val = 0
for i in range(12):
    for j in range(12):
        x = float(input())
        if j == c: sum_val += x
print(f"{sum_val if t == 'S' else sum_val / 12:.1f}")
```

NQ051 简单斐波那契 AcWing 717

"我突然想——能不能只算第N项，不输出前面的？"小鲁问。"当然可以！"小华说，"只需要把输出改为只输出最后一个值，中间结果不用存储。这就是O(1)空间复杂度——不管N多大，你只需要两个变量a和b。"

输入 一个整数N。

输出 第N项的值。

范围 $0 \leq N \leq 60$

输入

4

输出

3

思路 Py:a,b=0,1;for _ in range(n):a,b=b,a+b;print(a)。循环n次后a就是第n项。注意循环次数和输出的对应关系——这是递推的"根"模式。

技巧 O(1)空间的意思是：无论N=10还是N=60，内存用量恒定（只需2个变量）。这展示了递推相比递归的最大优势——不需要存储所有历史结果。

C++

```
#include <iostream>

using namespace std;

int main()
{
    int n;
    cin >> n;

    int a = 0, b = 1;

    for (int i = 0; i < n; i++)
    {
        cout << a << ' ';
        int c = a + b;
        a = b;
        b = c;
    }

    cout << endl;

    return 0;
}
```

Python

```
n = int(input())
a, b = 0, 1
for _ in range(n):
    a, b = b, a + b
print(a)
```

NQ052 数字序列和它的和 AcWing 722

小鲁在练习while循环的用法。"如果我想一直读数据直到遇到0为止怎么办？"小华说："while True:读入→判断→break。这是处理'不定长输入'的标准模式。结合for循环生成序列和累加，你就能处理各种输入场景了。"

输入 多行，每行两个整数M和N。以 $M \leq 0$ 或 $N \leq 0$ 结束。

输出 对每对M,N，输出所有整数和"Sum=X"。

范围 $M, N \leq 100$

输入

5 10
2 3
0 0

输出

5 6 7 8 9 10 Sum=45
2 3 Sum=5

思路 while+break是不定长输入的标准处理模式。对每对M,N，先交换确保 $M \leq N$ ，然后用for循环从M输出到N并累加。Python的range()+sum()组合最优雅。

技巧 Python: `print(' '.join(map(str, range(m, n+1))), f'Sum={sum(range(m, n+1))}')`。一行搞定！

C++

```
#include <iostream>
#include <algorithm>

using namespace std;

int main()
{
    int n, m;
    while (cin >> n >> m, n > 0 && m > 0)
    {
        if (n > m) swap(n, m);

        int sum = 0;
        for (int i = n; i <= m; i++)
        {
            cout << i << ' ';
            sum += i;
        }

        cout << "Sum=" << sum << endl;
    }

    return 0;
}
```

Python

```
while True:
    m, n = map(int, input().split())
    if m <= 0 or n <= 0: break
    if m > n: m, n = n, m
    nums = range(m, n + 1)
    print(' '.join(map(str, nums)),
          f"Sum={sum(nums)}")
```

NQ053 完全数 AcWing 725

小华在图书馆看到一本书上写着“完全数”。“ $6=1+2+3$ ， $28=1+2+4+7+14$ ……这些数太美了！”他兴奋地对小鲁说：“你知道吗？在 10^8 以内只有四个完全数：6,28,496,8128。这是数学家欧几里得和欧拉证明的。所以说——有时候数学知识可以让你少写很多代码！”

输入

一个整数N。

输出

每行一个完全数。

范围

 $1 \leq N \leq 10^8$

输入

30

输出

6

28

思路 10^8 以内只有4个完全数。直接判断它们是否 $\leq N$ 即可。如果用朴素算法（枚举因子求和），对于 $N=10^8$ 需要几十秒——而用数学知识， $O(1)$ 时间解决。数学+算法=效率的天花板。

技巧 这是“先思考，再编码”的典范。编程不是蛮力计算——有时候知道答案比计算答案更高效。厦门大学数学系有深厚的数论传统——学编程时别忘了你的数学课！

C++

```
#include <cstdio>
#include <iostream>

using namespace std;

int main()
{
    int n;
    cin >> n;

    while (n -- )
    {
        int x;
        cin >> x;

        int s = 0;
        for (int i = 1; i * i <= x; i ++ )
            if (x % i == 0)
            {
                if (i < x) s += i;
                if (i != x / i && x / i < x) s += x / i;
            }

        if (s == x) printf("%d is perfect\n", x);
        else printf("%d is not perfect\n", x);
    }

    return 0;
}
```

Python

```
n = int(input())
for x in [6, 28, 496, 8128]:
    if x <= n:
        print(x)
```

NQ054 质数 AcWing 726

小嘉路过实验室，看到小鲁和伙伴们正在讨论质数判定。"质数呀！"小嘉饶有兴趣地说，"我在南洋做生意的时候，数学帮我解决了不少问题。你们知道吗——判断一个数是不是质数，不需要检查到 $N-1$ ，只需要检查到 $\sqrt{N}$ 。因为如果 N 有大于 $\sqrt{N}$ 的因子，一定有一个小于 $\sqrt{N}$ 的对应因子。"

输入 一个整数 N 。

输出 每行一个质数。

范围 $2 \leq N \leq 10^6$

输入	输出
10	2
	3
	5
	7

思路 $\sqrt{N}$ 优化：对2到 N 的每个数 i ，只需检查2到 $\sqrt{i}$ 的因子。 $j*j \leq i$ 作循环条件比 $j \leq \text{sqrt}(i)$ 更高效（避免浮点运算）。时间复杂度从 $O(n^2)$ 降到 $O(n\sqrt{n})$ 。算法优化的第一课——数学知识直接转化为效率提升。

技巧 Py用`math.isqrt(i)`获得整数平方根。`j*j<=i`替代`j<=sqrt(i)`避免浮点。小嘉说得对： $\sqrt{N}$ 的数学原理是优化算法的关键。厦大校训“自强不息”——算法优化永无止境。

C++

```
#include <cstdio>
#include <iostream>

using namespace std;

int main()
{
    int n;
    cin >> n;
    while (n -- )
    {
        int p;
        cin >> p;

        bool is_prime = true;
        for (int i = 2; i * i <= p; i ++ )
            if (p % i == 0)
            {
                is_prime = false;
                break;
            }
        if (is_prime) printf("%d is prime\n", p);
        else printf("%d is not prime\n", p);
    }

    return 0;
}
```

Python

```
import math
n = int(input())
for i in range(2, n + 1):
    prime = True
    for j in range(2, int(math.sqrt(i)) + 1):
        if i % j == 0:
            prime = False
            break
    if prime:
        print(i)
```

本章知识点总结

- 数组五大操作：遍历(iterate)、筛选(filter)、变换(transform)、翻转(reverse)、查找(search)
- C++ vs Python：定长栈数组 vs 动态堆列表。C++更快，Python更灵活
- 递推思想：斐波那契——每个新值由前两个已知值计算，是DP的萌芽
- 算法优化第一课：质数判定的 $\sqrt{N}$ 优化——用数学将复杂度从 $O(n^2)$ 降到 $O(n\sqrt{n})$
- 厦大校训：自强不息，止于至善。编程之路漫长，但每一步都在进步